Attendance Requirement Statement for CS 156a Discussion Sessions

Please complete & attach as the 1st page of your submitted final set

[required for the final set to be graded and credited]

Name: Om Sanan

I have attended at least 8 of the 18 discussion sessions in class (at least 7 if I submitted the bonus exercise).

Signature: Om Sanan

Fall 2025

CS/CNS/EE 156a Learning Systems
Caltech - Fall 2025
https://caltech.instructure.com/courses/8882
(Learning From Data campus version)

Final Set

Due **Friday**, December 5, 2025, at 2:00 PM PST

*Definitions and notation follow the lectures. All questions have multiple-choice answers (***[a], [b], [c], ...***). Collaboration is not allowed in this final set. You can ask questions on the course forum for clarification only.* **You cannot discuss the problems with anyone**. *The open book/chatGPT policies in the initial handout apply to the final set as well.*

Note about the final set

- Be sure to give yourself enough time to complete this final set. There are twice as many problems in this set as there are in a homework set, and some problems require packages that will need time to get to work properly. A bit more time is given for this set compared to the homework (due date is Friday of next week).
- Problems cover different parts of the course. You are encouraged to consult your previous course work. To facilitate your search for relevant parts, an indexed version of the lecture video segments can be found at the Machine Learning Video Library:

  http://work.caltech.edu/library

- Answer each question by deriving the answer (carries 6 points) then selecting from the multiple-choice answers (carries 4 points). You can select 1 or 2 of the multiple-choice answers for each question, but you will get 4 or 2 points, respectively, for a correct answer. See the initial "**Course Description and Policies**" handout for important details.
- This is a no-collaboration set. You can only ask questions for clarification on the discussion forum, but otherwise you should not discuss the problems at all with anyone before the final set is due.

© 2012-2025 Yaser Abu-Mostafa. All rights reserved. No redistribution in any format. No translation or derivative products without written permission.

- **Nonlinear transforms**

1. The polynomial transform of order $Q = 10$ applied to $\mathcal{X}$ of dimension $d = 2$ results in a $\mathcal{Z}$ space of what dimensionality (not counting the constant coordinate $x_0 = 1$ or $z_0 = 1$)?

$$N = \binom{Q+d}{d} = \binom{12}{2} = 66$$

This is also:

$$\sum_{k=0}^{10} (k+1) = \frac{11 \cdot 12}{2} = 66$$

Without constant 66-1=65

[a] 12

[b] 20

[c] 35

[d] 100

[e] None of the above

- **Bias and Variance**

2. Recall that the average hypothesis $\bar{g}$ was based on training the same model $\mathcal{H}$ on different data sets $\mathcal{D}$ to get $g^{(\mathcal{D})} \in \mathcal{H}$, and taking the expected value of $g^{(\mathcal{D})}$ w.r.t. $\mathcal{D}$ to get $\bar{g}$. Which of the following models $\mathcal{H}$ could result in $\bar{g} \notin \mathcal{H}$?

[a] A singleton $\mathcal{H}$ ($\mathcal{H}$ has one hypothesis) The average is just h itself

[b] $\mathcal{H}$ is the set of all constant, real-valued hypotheses Avg of constant numbers is a constant

[c] $\mathcal{H}$ is the linear regression model linear functions are closed under average -> still linear

[d] $\mathcal{H}$ is the logistic regression model an avg of sigmoids of w's of different w's it not a sigmoid of some generic w'; rather an anv of logistic functions with different weights

[e] None of the above

Function has form: $g^{(D)}(x) = \frac{1}{1+e^{-w^T x}}$ Then, $\bar{g}(x) = E_D\left[\frac{1}{1+e^{-w^T x}}\right]$

- **Overfitting**

3. Which of the following statements is *false*?

[a] If there is overfitting, there must be two or more hypotheses that have different values of E_{in}.

[b] If there is overfitting, there must be two or more hypotheses that have different values of E_{out}.

[c] If there is overfitting, there must be two or more hypotheses that have different values of $(E_{\text{out}} - E_{\text{in}})$.

[d] We can always determine if there is overfitting by comparing the values of $(E_{\text{out}} - E_{\text{in}})$.

[e] We cannot determine overfitting based on one hypothesis only.

D is false. We can have a scenario where (E_{out} - E_{in}) is large but no other hypothesis in the set could have done better on E_{out}

In this case, we haven't really overfit relative to the available options, rather we just have some hard problem or sparse data.

It is also essential to know the relative ordering of E_{in} and E_{out} between hypothesis.

Stochastic noise comes from the inherent randomness in the data generation process P(y|x) and must exist no matter the hypothesis set that we choose as it's a property of the data/environment and not the model we choose.

4. Which of the following statements is true?

[a] Deterministic noise cannot occur with stochastic noise.

[b] Deterministic noise does not depend on the hypothesis set.

[c] Deterministic noise does not depend on the target function.

[d] Stochastic noise does not depend on the hypothesis set.

[e] Stochastic noise does not depend on the target distribution.

- **Regularization**

5. The regularized weight $\mathbf{w}_{\text{reg}}$ is a solution to:

$$\text{minimize} \quad \frac{1}{N}\sum_{n=1}^{N}(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n - y_n)^2 \quad \text{subject to} \quad \mathbf{w}^{\mathrm{T}}\Gamma^{\mathrm{T}}\Gamma\mathbf{w} \leq C,$$

where Γ is a Tikhonov matrix. If $\mathbf{w}_{\text{lin}}^{\mathrm{T}}\Gamma^{\mathrm{T}}\Gamma\mathbf{w}_{\text{lin}} \leq C$, where $\mathbf{w}_{\text{lin}}$ is the linear regression solution, then what is $\mathbf{w}_{\text{reg}}$?

[a] $\mathbf{w}_{\text{reg}} = \mathbf{w}_{\text{lin}}$

[b] $\mathbf{w}_{\text{reg}} = \Gamma\mathbf{w}_{\text{lin}}$

[c] $\mathbf{w}_{\text{reg}} = \Gamma^{\mathrm{T}}\Gamma\mathbf{w}_{\text{lin}}$

[d] $\mathbf{w}_{\text{reg}} = C\Gamma\mathbf{w}_{\text{lin}}$

[e] $\mathbf{w}_{\text{reg}} = C\mathbf{w}_{\text{lin}}$

If the unconstrained solution w_{lin} satisfies the constraint, then the constraint is inactive

When a constraint is inactive, the constrained solution equals the unconstrained solution

The given condition tells us what w_{lin} satisfied the constraint. Since w_{lin} minimizes objective function and it satisfies the constraint, it the optimal solution

6. Soft-order constraints that regularize polynomial models can be

Self-reminder: Soft-order constraints use regularization to penalize complexity without hard limits

[a] written as hard-order constraints

[b] translated into augmented error

[c] determined from the value of the VC dimension

[d] used to decrease both E_{in} and E_{out}

[e] None of the above is true

A constrained optimization problem

$\min_w E_{in}(w)$ subject to $w^T w \leq C$

can be translated (via Lagrange) to an unconstrained form:

$$\min_w E_{in}(w) + \lambda w^T w$$

Basically adding a penalty term to the error

Augmented error: $E_{aug}(w) = E_{in}(w) + \frac{\lambda}{N} w^T w$

- **Regularized Linear Regression**

We are going to experiment with linear regression for classification on the processed US Postal Service Zip Code data set from Homework 8. Download the data (extracted features of intensity and symmetry) for training and testing:

http://www.amlbook.com/data/zip/features.train

http://www.amlbook.com/data/zip/features.test

(the format of each row is: **digit intensity symmetry**). We will train two types of binary classifiers; one-versus-one (one digit is class $+1$ and another digit is class -1, with the rest of the digits disregarded), and one-versus-all (one digit is class $+1$ and the rest of the digits are class -1). When evaluating E_{in} and E_{out}, use binary classification error. Implement the regularized least-squares linear regression for classification that minimizes

$$\frac{1}{N}\sum_{n=1}^{N}\left(\mathbf{w}^{\mathrm{T}}\mathbf{z}_n - y_n\right)^2 \;+\; \frac{\lambda}{N}\mathbf{w}^{\mathrm{T}}\mathbf{w}$$

where $\mathbf{w}$ includes w_0.

7. Set $\lambda = 1$ and do not apply a feature transform (i.e., use $\mathbf{z} = \mathbf{x} = (1, x_1, x_2)$). Which among the following classifiers has the lowest E_{in}?

[**a**] 5 versus all

[**b**] 6 versus all

[**c**] 7 versus all

[**d**] 8 versus all

[**e**] 9 versus all

8. Now, apply a feature transform $\mathbf{z} = (1, x_1, x_2, x_1x_2, x_1^2, x_2^2)$, and set $\lambda = 1$. Which among the following classifiers has the lowest E_{out}?

[**a**] 0 versus all

[**b**] 1 versus all

[**c**] 2 versus all

[**d**] 3 versus all

[**e**] 4 versus all

9. If we compare using the transform versus not using it, and apply that to '0 versus all' through '9 versus all', which of the following statements is correct for $\lambda = 1$?

[**a**] Overfitting always occurs when we use the transform.

[**b**] The transform always improves the out-of-sample performance by at least 5% (E_{out} with transform $\leq 0.95E_{\text{out}}$ without transform).

[**c**] The transform does not make any difference in the out-of-sample performance.

[d] The transform always worsens the out-of-sample performance by at least 5%.

[e] The transform improves the out-of-sample performance of '5 versus all,' but by less than 5%.

10. Train the '1 versus 5' classifier with $\mathbf{z} = (1, x_1, x_2, x_1x_2, x_1^2, x_2^2)$ with $\lambda = 0.01$ and $\lambda = 1$. Which of the following statements is correct?

[a] Overfitting occurs (from $\lambda = 1$ to $\lambda = 0.01$).

[b] The two classifiers have the same E_{in}.

[c] The two classifiers have the same E_{out}.

[d] When λ goes up, both E_{in} and E_{out} go up.

[e] When λ goes up, both E_{in} and E_{out} go down.

- **Support Vector Machines**

11. Consider the following training set generated from a target function $f : \mathcal{X} \to \{-1, +1\}$ where $\mathcal{X} = \mathbb{R}^2$

$$\begin{array}{ccc} \mathbf{x}_1 = (1,0), y_1 = -1 & \mathbf{x}_2 = (0,1), y_2 = -1 & \mathbf{x}_3 = (0,-1), y_3 = -1 \\ \mathbf{x}_4 = (-1,0), y_4 = +1 & \mathbf{x}_5 = (0,2), y_5 = +1 & \mathbf{x}_6 = (0,-2), y_6 = +1 \\ & \mathbf{x}_7 = (-2,0), y_7 = +1 & \end{array}$$

Transform this training set into another two-dimensional space $\mathcal{Z}$

$$z_1 = x_2^2 - 2x_1 - 1 \qquad z_2 = x_1^2 - 2x_2 + 1$$

Using geometry (not quadratic programming), what values of $\mathbf{w}$ (without w_0) and b specify the separating plane $\mathbf{w}^{\mathrm{T}}\mathbf{z} + b = 0$ that maximizes the margin in the $\mathcal{Z}$ space? The values of w_1, w_2, b are:

[a] $-1,\ 1,\ -0.5$

[b] $1,\ -1,\ -0.5$

[c] $1,\ 0,\ -0.5$

[d] $0,\ 1,\ -0.5$

[e] None of the above would work.

x_1=(1, 0), y_1=-1; z_1=0^2 - 2(1) - 1 = -3; z_2 = 1^2 - 2(0) + 1 = 2; z_1 = (-3, 2); y_1 = -1

x_2 = (0, 1), y_2 = -1 -> z_2 = (0, -1)

x_3 = (0, -1), y_3 = -1 -> z_3 = (0, 3)

x_4 = (-1, 0), y_4 = 1 -> z_4 = (1, 2)

x_5 = (0, 2), y_5 = 1 -> z_2 = (3, -3)

x_6 = (0, -2), y_6 = 1 -> z_6 = (3, 5)

x_7 = (-2, 0), y_7 = 1 -> z_7 = (3, 5)

Negative class: (-3, 2), (0, -1), (0, 3)

Positive class: (1, 2), (3, -3), (3, 5), (3, 5)

Geometrically these are perfectly separated along the z_1 axis: all negatives have $z_1 \le 0$; all positives have $z_1 \ge 1$

The natural max-margin separator is the vertical line midway between 0 and 1 at 0.5

This corresponds to a hyperplane

$w^T z + b = 0 \to 1 \cdot z_1 + 0 \cdot z_2 - 0.5 = 0$

So w = (1, 0), b = -0.5

This makes sense because margins $y_i(w^T z_i + b)$ are positive for all points and closest ones are at distance 0.5 on each side to maximize the margin

12. Consider the same training set of the previous problem, but instead of explicitly transforming the input space $\mathcal{X}$, apply the hard-margin SVM algorithm with the kernel

$$K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^{\mathrm{T}}\mathbf{x}')^2$$

(which corresponds to a second-order polynomial transformation). Set up the expression for $\mathcal{L}(\alpha_1 ... \alpha_7)$ and solve for the optimal $\alpha_1, ..., \alpha_7$ (numerically, using a quadratic programming package). The number of support vectors you get is in what range?

[a] 0-1

[b] 2-3

[c] 4-5

[d] 6-7

[e] >7

• Radial Basis Functions

We experiment with the RBF model, both in regular form (Lloyd + pseudo-inverse) with K centers:

$$\text{sign}\left(\sum_{k=1}^{K} w_k \ \exp\left(-\gamma \ ||\mathbf{x} - \mu_k||^2\right) + \ b \right)$$

(notice that there is a bias term), and in kernel form (using the RBF kernel in hard-margin SVM):

$$\text{sign}\left(\sum_{\alpha_n > 0} \alpha_n y_n \ \exp\left(-\gamma \ ||\mathbf{x} - \mathbf{x}_n||^2\right) + \ b \right).$$

The input space is $\mathcal{X} = [-1, 1] \times [-1, 1]$ with uniform probability distribution, and the target is

$$f(\mathbf{x}) = \text{sign}(x_2 - x_1 + 0.25 \sin(\pi x_1))$$

which is slightly nonlinear in the $\mathcal{X}$ space. In each run, generate 100 training points at random using this target, and apply both forms of RBF to these training points. Here are some guidelines:

- Repeat the experiment for as many runs as needed to get the answer to be stable (statistically away from flipping to the closest competing answer).
- In case a data set is not separable in the '$\mathcal{Z}$ space' by the RBF kernel using hard-margin SVM, discard the run but keep track of how often this happens, if ever.

- When you use Lloyd's algorithm, initialize the centers to random points in $\mathcal{X}$ and iterate until there is no change from iteration to iteration. If a cluster becomes empty, discard the run and repeat.

13. For $\gamma = 1.5$, how often do you get a data set that is not separable by the RBF kernel (using hard-margin SVM)? *Hint: Run the hard-margin SVM, then check if the solution has* $E_{\text{in}} \neq 0$.

[a] $\leq 5\%$ of the time

[b] $> 5\%$ but $\leq 10\%$ of the time

[c] $> 10\%$ but $\leq 20\%$ of the time

[d] $> 20\%$ but $\leq 40\%$ of the time

[e] $> 40\%$ of the time

14. If we use $K = 9$ for regular RBF and take $\gamma = 1.5$, how often does the kernel form beat the regular form (excluding runs mentioned in Problem 13 and runs with empty clusters, if any) in terms of E_{out}?

[a] $\leq 15\%$ of the time

[b] $> 15\%$ but $\leq 30\%$ of the time

[c] $> 30\%$ but $\leq 50\%$ of the time

[d] $> 50\%$ but $\leq 75\%$ of the time

[e] $> 75\%$ of the time

15. If we use $K = 12$ for regular RBF and take $\gamma = 1.5$, how often does the kernel form beat the regular form (excluding runs mentioned in Problem 13 and runs with empty clusters, if any) in terms of E_{out}?

[a] $\leq 10\%$ of the time

[b] $> 10\%$ but $\leq 30\%$ of the time

[c] $> 30\%$ but $\leq 60\%$ of the time

[d] $> 60\%$ but $\leq 90\%$ of the time

[e] $> 90\%$ of the time

16. Now we focus on regular RBF only, with $\gamma = 1.5$. If we go from $K = 9$ clusters to $K = 12$ clusters (only 9 and 12), which of the following 5 cases happens most often in your runs (excluding runs with empty clusters, if any)? Up or down means strictly so.

[a] E_{in} goes down, but E_{out} goes up.

[b] E_{in} goes up, but E_{out} goes down.

[c] Both E_{in} and E_{out} go up.

[d] Both E_{in} and E_{out} go down.

[e] E_{in} and E_{out} remain the same.

17. For regular RBF with $K = 9$, if we go from $\gamma = 1.5$ to $\gamma = 2$ (only 1.5 and 2), which of the following 5 cases happens most often in your runs (excluding runs with empty clusters, if any)? Up or down means strictly so.

[a] E_{in} goes down, but E_{out} goes up.

[b] E_{in} goes up, but E_{out} goes down.

[c] Both E_{in} and E_{out} go up.

[d] Both E_{in} and E_{out} go down.

[e] E_{in} and E_{out} remain the same.

18. What is the percentage of time that regular RBF achieves $E_{\text{in}} = 0$ with $K = 9$ and $\gamma = 1.5$ (excluding runs with empty clusters, if any)?

[a] $\leq 10\%$ of the time

[b] $> 10\%$ but $\leq 20\%$ of the time

[c] $> 20\%$ but $\leq 30\%$ of the time

[d] $> 30\%$ but $\leq 50\%$ of the time

[e] $> 50\%$ of the time

- **Aggregation**

19. Given two learned hypotheses g_1 and g_2, we construct the aggregate hypothesis g given by $g(\mathbf{x}) = \frac{1}{2}\left(g_1(\mathbf{x}) + g_2(\mathbf{x})\right)$ for all $\mathbf{x} \in \mathcal{X}$. If we use mean-squared error, which of the following statements is true?

[a] $E_{\text{out}}(g)$ cannot be worse than $E_{\text{out}}(g_1)$.

[b] $E_{\text{out}}(g)$ cannot be worse than the smaller of $E_{\text{out}}(g_1)$ and $E_{\text{out}}(g_2)$.

[c] $E_{\text{out}}(g)$ cannot be worse than the average of $E_{\text{out}}(g_1)$ and $E_{\text{out}}(g_2)$.

[d] $E_{\text{out}}(g)$ has to be between $E_{\text{out}}(g_1)$ and $E_{\text{out}}(g_2)$ (including the end values of that interval).

[e] None of the above

See next page.

- **Hints**

20. Let the target function $f : R^d \to R$ be the credit line based on the customer data $\mathbf{x}$, and assume $f(\mathbf{x})$ is monotonically non-decreasing in the annual salary, one of the variables in the input $\mathbf{x}$. If the transformation of $\mathbf{x}$ that increases the salary variable and keeps the rest of the variables in $\mathbf{x}$ unchanged leads to a transformed input $\mathbf{x}'$, which of the following error measures applied to a virtual (unlabeled) example $(\mathbf{x}, \mathbf{x}')$ would capture enforcing the hint on that example?

[a] $g(\mathbf{x}) - g(\mathbf{x}')$

[b] $g(\mathbf{x}') - g(\mathbf{x})$

[c] $(\ g(\mathbf{x}) - g(\mathbf{x}')\ )^2$

[d] $\mathrm{ReLU}(\ g(\mathbf{x}) - g(\mathbf{x}')\ )$

[e] $\mathrm{ReLU}(\ g(\mathbf{x}') - g(\mathbf{x})\ )$

19. MSE

$$E_{out}(h) = E\left[h(x) - f(x)^2\right]$$

$$E_{out}(h) = E\left[\left(\frac{g_1(x) + g_2(x)}{2} - f(x)\right)^2\right]$$

$$E_{out}(h) = E\left[\left(\frac{g_1(x) - f_1(x)}{2} + \frac{g_2(x) - f_2(x)}{2}\right)^2\right]$$

We can use Jensen's inequality:

$$\phi(\lambda a + (1-\lambda) b) \leq \lambda \phi(a) + (1-\lambda)\phi(b)$$

Using lambda = 0.5:

$$\left(\frac{a+b}{2}\right)^2 \leq \frac{a^2}{2} + \frac{b^2}{2}$$

Let a=g_1(x)-f(x) and b=g_2(x)-f(x). Then:

$$(g(x) - f(x))^2 \leq \frac{1}{2}(g_1(x) - f(x))^2 + \frac{1}{2}(g_2(x) - f(x))^2$$

$$E\left[(g(x) - f(x))^2\right] \leq \frac{1}{2} E\left[(g_1(x) - f(x))^2 + (g_2(x) - f(x))^2\right]$$

$$E_{out}(g) \leq \frac{E_{out}(g_1) + E_{out}(g_2)}{2}$$

Therefore error of the average hypothesis is less than or equal to the average of the individual errors.

20.

We want to define some error penalty that forces the hypothesis g to respect the monotonic property of the target function.

If x' has as a higher salary than x (and all other features are equal), then the credit line should not decrease

Target: f(x') ≥ f(x)

A voilation occurs if g(x') < g(x) or g(x) - g(x') > 0

Error measure:

- g(x') - g(x) > 0 -> positive error
- g(x') - g(x) ≤ 0 -> negative error

This aligns with [d] because ReLU(g(x) - g(x')) = max(0, g(x) - g(x')) which excactly satisfies our defined error measure.

Problems 7-10

```
import numpy as np
import os
import urllib.request

def load_data(filename):
    """
    loads data from file
    """
    data = np.loadtxt(filename)
    # first column is digit (y), next two are features (X)
    y_raw = data[:, 0]
    X_raw = data[:, 1:]
    return X_raw, y_raw

def transform(X_raw):
    """
    apply nonlinear transformation for P8-10:
    z = (1, x_1, x_2, x_1*x_2, x_1^2, x_2^2)
    """
    x1 = X_raw[:, 0]
    x2 = X_raw[:, 1]

    # get N
    N = X_raw.shape[0]

    # transformed feature matrix
    Z = np.c_[np.ones(N), x1, x2, x1 * x2, x1**2, x2**2]

    return Z

def transform_linear(X_raw):
    """
    apply simple linear transformation (bias only) P7:
    z = (1, x_1, x_2)
    """
    # get N
    N = X_raw.shape[0]

    # create feature matrix with bias
    Z = np.c_[np.ones(N), X_raw]
    return Z

def get_binary_labels(y_raw, digit_pos, digit_neg=None):
    """
    converts raw digits to binary labels +1/-1
    """
```

```python
    if digit_neg is None:
        # 1vall
        y = np.where(y_raw == digit_pos, 1.0, -1.0)
        return y
    else:
        # 1v1
        pass

def calculate_classification_error(Z, y, w):
    """
    calculates classification error
    """
    # prediction scores
    y_scores = Z @ w

    # classify based on the score sign
    y_pred = np.sign(y_scores)

    # edge case -> sign(0) = 0; assign to -1
    y_pred[y_pred == 0] = -1

    # fraction of misclassified points
    misclassified_count = np.sum(y_pred != y)
    error_rate = misclassified_count / len(y)

    return error_rate

def solve_ridge_regression(Z, y, lambda_val):
    """
    solve ridge regression:
    w = (Z^T Z + lambda I)^-1 Z^T y
    """
    N, d = Z.shape
    I = np.eye(d)

    # compute A = Z^T Z + lambda * I
    A = Z.T @ Z + lambda_val * I

    # compute b = Z^T y
    b = Z.T @ y

    # solve Aw = b
    w = np.linalg.solve(A, b)
    return w

if __name__ == "__main__":
    X_train_raw, y_train_raw = load_data('./features.train.txt')
    X_test_raw, y_test_raw = load_data('./features.test.txt')

    print(f"Training set size: {len(y_train_raw)}")
```

```
    print(f"Test set size: {len(y_test_raw)}")

    # P7
    print("\nProblem 7")
    print("One v All, No Transform (Linear), lambda=1")

    digits_q7 = [5, 6, 7, 8, 9]
    ein_results = {}

    Z_train_lin = transform_linear(X_train_raw)

    for digit in digits_q7:
        # make labels: digit v all
        y_train = np.where(y_train_raw == digit, 1.0, -1.0)

        # train ridge
        w = solve_ridge_regression(Z_train_lin, y_train, lambda_val=1)

        # calculate Ein
        e_in = calculate_classification_error(Z_train_lin, y_train, w)
        ein_results[digit] = e_in

        print(f"Digit {digit}: E_in = {e_in:.5f}")

    best_digit_q7 = min(ein_results, key=ein_results.get)
    print(f"Lowest E_in: Digit {best_digit_q7}")

    # P8
    print("\nProblem 8")
    print("One v All, Poly Transform, lambda=1")

    digits_q8 = [0, 1, 2, 3, 4]
    eout_results = {}

    Z_train_poly = transform(X_train_raw)
    Z_test_poly = transform(X_test_raw)

    for digit in digits_q8:
        # prepare labels
        y_train = np.where(y_train_raw == digit, 1.0, -1.0)
        y_test = np.where(y_test_raw == digit, 1.0, -1.0)

        # train ridge
        w = solve_ridge_regression(Z_train_poly, y_train,
lambda_val=1)

        # calculate Eout
        e_out = calculate_classification_error(Z_test_poly, y_test, w)
        eout_results[digit] = e_out

        print(f"Digit {digit}: E_out = {e_out:.5f}")
```

```
    best_digit_q8 = min(eout_results, key=eout_results.get)
    print(f"Lowest E_out: Digit {best_digit_q8}")

    # P9
    print("\nProblem 9")
    print("No Transform vs Poly Transform")

    lambda_val = 1
    Z_test_lin = transform_linear(X_test_raw)

    for digit in range(10):
        # prepare labels
        y_train = np.where(y_train_raw == digit, 1.0, -1.0)
        y_test = np.where(y_test_raw == digit, 1.0, -1.0)

        # no transform
        w_lin = solve_ridge_regression(Z_train_lin, y_train,
lambda_val)
        e_out_no = calculate_classification_error(Z_test_lin, y_test,
w_lin)

        # poly transform
        w_poly = solve_ridge_regression(Z_train_poly, y_train,
lambda_val)
        e_out_poly = calculate_classification_error(Z_test_poly,
y_test, w_poly)

        print(f"Digit {digit}: E_out(NoTrans)={e_out_no:.4f},
E_out(Trans)={e_out_poly:.4f}")

        if digit == 5:
            ratio = e_out_poly / e_out_no
            print(f"  Digit 5 Ratio (Trans/NoTrans): {ratio:.4f}")

    # P10
    print("\nProblem 10")
    print("One v One (1 vs 5), Poly Transform")

    # filter data for 1 vs 5
    # we need to extract rows where raw label is 1 or 5
    mask_train = (y_train_raw == 1) | (y_train_raw == 5)
    mask_test = (y_test_raw == 1) | (y_test_raw == 5)

    X_train_1v5 = X_train_raw[mask_train]
    y_train_1v5_raw = y_train_raw[mask_train]

    X_test_1v5 = X_test_raw[mask_test]
    y_test_1v5_raw = y_test_raw[mask_test]
```

```
    # set labels: 1 is +1, 5 is -1
    y_train_1v5 = np.where(y_train_1v5_raw == 1, 1.0, -1.0)
    y_test_1v5 = np.where(y_test_1v5_raw == 1, 1.0, -1.0)

    # transform filtered data
    Z_train_1v5 = transform(X_train_1v5)
    Z_test_1v5 = transform(X_test_1v5)

    # lambda = 1
    w_1 = solve_ridge_regression(Z_train_1v5, y_train_1v5,
lambda_val=1)
    ein_1 = calculate_classification_error(Z_train_1v5, y_train_1v5,
w_1)
    eout_1 = calculate_classification_error(Z_test_1v5, y_test_1v5,
w_1)

    # lambda = 0.01
    w_01 = solve_ridge_regression(Z_train_1v5, y_train_1v5,
lambda_val=0.01)
    ein_01 = calculate_classification_error(Z_train_1v5, y_train_1v5,
w_01)
    eout_01 = calculate_classification_error(Z_test_1v5, y_test_1v5,
w_01)

    print(f"Lambda=1: E_in={ein_1:.4f}, E_out={eout_1:.4f}")
    print(f"Lambda=0.01: E_in={ein_01:.4f}, E_out={eout_01:.4f}")
```

```
Training set size: 7291
Test set size: 2007

Problem 7
One v All, No Transform (Linear), lambda=1
Digit 5: E_in = 0.07626
Digit 6: E_in = 0.09107
Digit 7: E_in = 0.08847
Digit 8: E_in = 0.07434
Digit 9: E_in = 0.08833
Lowest E_in: Digit 8

Problem 8
One v All, Poly Transform, lambda=1
Digit 0: E_out = 0.10663
Digit 1: E_out = 0.02192
Digit 2: E_out = 0.09865
Digit 3: E_out = 0.08271
Digit 4: E_out = 0.09965
Lowest E_out: Digit 1

Problem 9
No Transform vs Poly Transform
```

```
Digit 0: E_out(NoTrans)=0.1151, E_out(Trans)=0.1066
Digit 1: E_out(NoTrans)=0.0224, E_out(Trans)=0.0219
Digit 2: E_out(NoTrans)=0.0987, E_out(Trans)=0.0987
Digit 3: E_out(NoTrans)=0.0827, E_out(Trans)=0.0827
Digit 4: E_out(NoTrans)=0.0997, E_out(Trans)=0.0997
Digit 5: E_out(NoTrans)=0.0797, E_out(Trans)=0.0792
  Digit 5 Ratio (Trans/NoTrans): 0.9938
Digit 6: E_out(NoTrans)=0.0847, E_out(Trans)=0.0847
Digit 7: E_out(NoTrans)=0.0732, E_out(Trans)=0.0732
Digit 8: E_out(NoTrans)=0.0827, E_out(Trans)=0.0827
Digit 9: E_out(NoTrans)=0.0882, E_out(Trans)=0.0882

Problem 10
One v One (1 vs 5), Poly Transform
Lambda=1: E_in=0.0051, E_out=0.0259
Lambda=0.01: E_in=0.0045, E_out=0.0283
```

Problem 12

```
import numpy as np
from cvxopt import matrix, solvers

# data (x1, x2,...,x7)
X = np.array([[1, 0], [0, 1], [0,-1], [-1,0], [0, 2], [0,-2], [-2,0]],
dtype=float)

y = np.array([-1, -1, -1, 1, 1, 1, 1], dtype=float)
N = len(y)

# Kernel K(x,x')=(1 + x^T x')^2
def kernel(x, xp):
    return (1 + np.dot(x, xp))**2

# Gram matrix
K = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        K[i,j] = kernel(X[i], X[j])

# Dual SVM QP:
# maximize 1^Tα − 1/2α^T (yy^T∘K)α
# subject to y^Tα = 0, α >= 0

# H=(yy^T) ∘ K
H = np.outer(y, y) * K
P = matrix(H)
q = matrix(-1 * np.ones(N))
```

```
# Equality constraint: y^T α = 0
A = matrix(y, (1, N))
b = matrix(0.0)

# Inequality: α >= 0 -> -Iα ≤ 0
G = matrix(-np.eye(N))
h = matrix(np.zeros(N))

# Solve
solvers.options['show_progress'] = False
solution = solvers.qp(P, q, G, h, A, b)
alpha = np.array(solution['x']).flatten()

sv_idx = np.where(alpha > 1e-6)[0]
print("Optimal alpha:")
print(alpha)
print("\nIndices:", sv_idx)
print("Num support vectors:", len(sv_idx))
```

```
Optimal alpha:
[4.32455513e-09 7.03703704e-01 7.03703704e-01 8.88888891e-01
 2.59259260e-01 2.59259260e-01 5.27081504e-10]

Indices: [1 2 3 4 5]
Num support vectors: 5
```

Problems 15-18

```
import numpy as np
from sklearn.svm import SVC
from scipy.spatial.distance import cdist

def target_function(X):
    x1 = X[:, 0]
    x2 = X[:, 1]
    vals = x2 - x1 + 0.25 * np.sin(np.pi * x1)
    y = np.sign(vals)
    y[y == 0] = 1
    return y

def generate_data(N):
    X = np.random.uniform(-1, 1, (N, 2))
    y = target_function(X)
    return X, y

def get_error_rate(y_true, y_pred):
    """
    binary classification error
```

```
    """
    return np.mean(y_true != y_pred)

def lloyd_clustering(X, K, max_iters=100):
    # centers in the input space
    centers = np.random.uniform(-1, 1, (K, 2))

    for _ in range(max_iters):
        # set points to nearest center
        dists = cdist(X, centers, metric='sqeuclidean')
        labels = np.argmin(dists, axis=1)

        new_centers = np.zeros_like(centers)
        empty_cluster = False

        for k in range(K):
            cluster_points = X[labels == k]
            if len(cluster_points) == 0:
                empty_cluster = True
                break
            new_centers[k] = np.mean(cluster_points, axis=0)

        if empty_cluster:
          # in this cade discard
            return None

        # convergence check
        if np.allclose(centers, new_centers):
            centers = new_centers
            break

        centers = new_centers

    return centers

def regular_rbf(X_train, y_train, centers, gamma):
    """
    Phi(x) = [exp(-gamma ||x - mu_1||^2), ..., exp(-gamma ||x - mu_K||
^2), 1]
    w = pinv(Phi) * y
    """
    # RBF features
    dists = cdist(X_train, centers, metric='sqeuclidean')
    Phi = np.exp(-gamma * dists)

    # bias term
    N = X_train.shape[0]
    Phi = np.hstack([Phi, np.ones((N, 1))])

    # solve for weights
```

```
    w = np.linalg.pinv(Phi) @ y_train

    # in sample predictios and error
    preds = np.sign(Phi @ w)
    preds[preds == 0] = 1
    ein = get_error_rate(y_train, preds)

    return w, ein

def predict_regular_rbf(X_test, w, centers, gamma):
    dists = cdist(X_test, centers, metric='sqeuclidean')
    Phi = np.exp(-gamma * dists)
    Phi = np.hstack([Phi, np.ones((X_test.shape[0], 1))])
    preds = np.sign(Phi @ w)
    preds[preds == 0] = 1
    return preds

def run_kernel_svm(X_train, y_train, gamma):
  # Hard margin
    clf = SVC(C=1e6, kernel='rbf', gamma=gamma)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_train)
    ein = get_error_rate(y_train, y_pred)
    return clf, ein

if __name__ == "__main__":
    np.random.seed(0)

    max_runs = 500

    stats = {'p13_inseparable': 0, 'p13_total': 0,
'p14_kernel_better': 0, 'p14_total': 0, 'p15_kernel_better': 0,
             'p15_total': 0, 'p16_counts': {'down_up': 0, 'up_down':
0, 'up_up': 0, 'down_down': 0, 'same': 0},
             'p16_total': 0, 'p17_counts': {'down_up': 0, 'up_down':
0, 'up_up': 0, 'down_down': 0, 'same': 0},
             'p17_total': 0, 'p18_ein_zero': 0, 'p18_total': 0
    }

    # make some large test set for stable E_out estimabillity
    X_test_global, y_test_global = generate_data(5000)

    gamma_base = 1.5
    gamma_alt = 2.0

    while stats['p14_total'] < max_runs:
        X, y = generate_data(100)

        svm_model, svm_ein = run_kernel_svm(X, y, gamma_base)
```

```
        # P13: separability check
        stats['p13_total'] += 1
        is_separable = (svm_ein == 0)
        if not is_separable:
            stats['p13_inseparable'] += 1

        # Compute SVM E_out only if separable (P14/P15)
        if is_separable:
            svm_pred_test = svm_model.predict(X_test_global)
            svm_eout = get_error_rate(y_test_global, svm_pred_test)

        centers_9 = lloyd_clustering(X, 9)
        if centers_9 is None:
            continue

        w_9, ein_9 = regular_rbf(X, y, centers_9, gamma_base)
        pred_9 = predict_regular_rbf(X_test_global, w_9, centers_9,
gamma_base)
        eout_9 = get_error_rate(y_test_global, pred_9)

        centers_12 = lloyd_clustering(X, 12)
        if centers_12 is None:
            continue

        w_12, ein_12 = regular_rbf(X, y, centers_12, gamma_base)
        pred_12 = predict_regular_rbf(X_test_global, w_12, centers_12,
gamma_base)
        eout_12 = get_error_rate(y_test_global, pred_12)

        w_9_g2, ein_9_g2 = regular_rbf(X, y, centers_9, gamma_alt)
        pred_9_g2 = predict_regular_rbf(X_test_global, w_9_g2,
centers_9, gamma_alt)
        eout_9_g2 = get_error_rate(y_test_global, pred_9_g2)


        # P14
        if is_separable:
            stats['p14_total'] += 1
            if svm_eout < eout_9:
                stats['p14_kernel_better'] += 1

        # P15
        if is_separable:
            stats['p15_total'] += 1
            if svm_eout < eout_12:
                stats['p15_kernel_better'] += 1

        # P16
        stats['p16_total'] += 1
        if ein_12 < ein_9 and eout_12 > eout_9:
```

```
            stats['p16_counts']['down_up'] += 1
        elif ein_12 > ein_9 and eout_12 < eout_9:
            stats['p16_counts']['up_down'] += 1
        elif ein_12 > ein_9 and eout_12 > eout_9:
            stats['p16_counts']['up_up'] += 1
        elif ein_12 < ein_9 and eout_12 < eout_9:
            stats['p16_counts']['down_down'] += 1
        else:
            stats['p16_counts']['same'] += 1

        # P17
        stats['p17_total'] += 1
        if ein_9_g2 < ein_9 and eout_9_g2 > eout_9:
            stats['p17_counts']['down_up'] += 1
        elif ein_9_g2 > ein_9 and eout_9_g2 < eout_9:
            stats['p17_counts']['up_down'] += 1
        elif ein_9_g2 > ein_9 and eout_9_g2 > eout_9:
            stats['p17_counts']['up_up'] += 1
        elif ein_9_g2 < ein_9 and eout_9_g2 < eout_9:
            stats['p17_counts']['down_down'] += 1
        else:
            stats['p17_counts']['same'] += 1

        # P18
        stats['p18_total'] += 1
        if np.isclose(ein_9, 0.0):
            stats['p18_ein_zero'] += 1


    # P13
    pct_inseparable = (stats['p13_inseparable'] / stats['p13_total'])
* 100
    print(f"P13: SVM Inseparable Amt: {pct_inseparable:.2f}% "
          f"= {stats['p13_inseparable']}/{stats['p13_total']}")

    # P14
    pct_kernel_better_9 = (stats['p14_kernel_better'] /
stats['p14_total']) * 100
    print(f"P14: Kernel beats Regular (K=9): {pct_kernel_better_9:.2f}
% "
          f"= {stats['p14_kernel_better']}/{stats['p14_total']}")

    # P15
    pct_kernel_better_12 = (stats['p15_kernel_better'] /
stats['p15_total']) * 100
    print(f"P15: Kernel beats Regular (K=12):
{pct_kernel_better_12:.2f}% "
          f"= {stats['p15_kernel_better']}/{stats['p15_total']}")

    # P16
```

```
    print("P16: K=9 -> K=12 (Gama=1.5):")
    for k, v in stats['p16_counts'].items():
        pct = (v / stats['p16_total']) * 100 if stats['p16_total'] > 0
else 0.0
        print(f". {k}: {v} ({pct:.2f}%)")

    # P17
    print("P17: Gamma=1.5 -> Gamma=2.0 (K=9):")
    for k, v in stats['p17_counts'].items():
        pct = (v / stats['p17_total']) * 100 if stats['p17_total'] > 0
else 0.0
        print(f". {k}: {v} ({pct:.2f}%)")

    # P18
    pct_ein_zero = (stats['p18_ein_zero'] / stats['p18_total']) * 100
    print(f"P18: Regular RBF (K=9, Gamma=1.5) Ein=0:
{pct_ein_zero:.2f}% "
          f"= {stats['p18_ein_zero']}/{stats['p18_total']}")
```

```
P13: SVM Inseparable Amt: 0.00% = 0/663
P14: Kernel beats Regular (K=9): 91.00% = 455/500
P15: Kernel beats Regular (K=12): 84.40% = 422/500
P16: K=9 -> K=12 (Gama=1.5):
. down_up: 44 (8.80%)
. up_down: 37 (7.40%)
. up_up: 38 (7.60%)
. down_down: 272 (54.40%)
. same: 109 (21.80%)
P17: Gamma=1.5 -> Gamma=2.0 (K=9):
. down_up: 23 (4.60%)
. up_down: 10 (2.00%)
. up_up: 174 (34.80%)
. down_down: 9 (1.80%)
. same: 284 (56.80%)
P18: Regular RBF (K=9, Gamma=1.5) Ein=0: 3.00% = 15/500
```